tpkg wire format (L0)

TBK SPEC 02:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 02:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. File anatomy 4

2. Trailer header — 166 bytes, always at the package end 5

3. Slot record — 280 bytes 6

4. v2 chain-of-trust extension (opt-in; flag bit1 `SIGNED_V2`) 6

5. The installability flag (bit2 `NO_INSTALL`) 7

5a. The quiet-notices flag (bit3 `QUIET_NOTICES`; tebako#400, spec 23 §14) 7

6. Orthogonality note (normative) 7

5b. Typed extension blocks (the L2 home; locked 2026-07-26) 8

6. Absent vs corrupt; validation 8

7. Errors (Rust `TpkgError` ↔ C `TPKG_ERR_*`, 1:1) 9

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Normative byte-level specification of the tebako package container. Source of truth: `crates/tpkg` (byte-exact with the reference C99 `include/tebako/tpkg.h`; golden vectors pin the layout). All v1 integers little-endian; the trailer header sits at the logical EOF (§1) at fixed size. The container knows **nothing** about runtimes, entrypoints, or languages — it carries bytes, slots, mounts, and a resolution hint. **Authenticated, signed, and encrypted packages:** authentication/signing is the v2 extension (§4); encryption is a TFS-layer transform of the slot images (spec 10) — the container itself stays byte-identical either way.

1. File anatomy

FIGURE 1

```
[payload bytes][slot table: n × 280][v2 extension?][trailer
header: 166 @ EOF]
```

"EOF" is the LOGICAL end of the package: the physical EOF, except on a package signed post-press. On a Mach-O package codesigned post-press (spec 31 §1.2), codesign appends the code-signature superblob after the trailer, 16-byte aligned with a zero pad. Readers locate the trailer end via the tail slice's LC_CODE_SIGNATURE: when the superblob reaches the physical EOF exactly, the trailer header is the self-validating (magic + crc + bounds) window ending at the superblob's dataoff or at most 15 zero pad bytes earlier; every other shape reads at the physical EOF. On a PE package Authenticode-signed post-press (spec 34 §1.2/§6), the signer appends the WIN_CERTIFICATE table after the trailer and points the security data directory (`IMAGE_DIRECTORY_ENTRY_SECURITY`, a file offset,

© 2026 The Tebako Project

not an RVA) at it: when the certificate table reaches the physical EOF exactly, the trailer header is the same self-validating window ending at the table's start or at most 15 zero pad bytes earlier (the table is 8-byte aligned); an absent or zero-length directory entry means unsigned. Writers never produce the suffixed shape — only the platform signer does, and the container bytes it hashes are unchanged by it.

2. Trailer header — 166 bytes, always at the package end

(The package end is the physical EOF, or — on a post-press-signed Mach-O/PE — just before the trailing superblob's or certificate table's alignment pad; §1.)

TABLE 1

offset	size	field
0	10	magic `"TEBAKOTFS\0"`
10	4	u32 version = 1 (extensions ride flags, not version bumps)
14	4	u32 package_flags (bit0 LEAN, bit1 SIGNED_V2, bit2 NO_INSTALL, bit3 QUIET_NOTICES)
18	4	u32 slot_count (1..=8)
22	8	u64 slot_table_offset (absolute file offset of slot 0)
30	128	char runtime_ref[128] — resolution hint (spec 05 §1); empty = classic
158	4	u32 launcher_abi
162	4	u32 crc32 over [0,162) — accident integrity, NOT authenticity

`TPKG_MAGIC_PREFIX_LEN` = 4 (`"TEBA"`) discriminates absent vs corrupt.

Bit0's *name* is the only deprecated thing about it (locked 2026-08-25, tebako#460): read LEAN = 1 as "the package does not carry its runtime pair" and LEAN = 0 as "the runtime pair is carried as ordinary slots" — the *shared-runtime* and *self-contained* presets of spec 23 §13.2 (whose `lean`/`fat` aliases exist only so older documents stay readable). The BIT itself stays load-bearing exactly as spec 18 S4 requires: flags are never skipped, and an unknown set bit is a named refusal.

© 2026 The Tebako Project

3. Slot record — 280 bytes

TABLE 2

offset	size	field
0	8	u64 offset (absolute file offset of the image)
8	8	u64 size
16	4	u32 format_id — IMAGE FORMAT ONLY: 0=auto, 1=dwarfs, 2=squashfs, 3=zip, 5=limnifs (spec 20, PLANNED) (4=runtime: legacy role wart, §5)
20	4	u32 flags (0 today; future home of role bits)
24	256	char mount_point[256] (UTF-8, NUL-padded)

4. v2 chain-of-trust extension (opt-in; flag bit1 `SIGNED_V2`)

Between slot table and header; all its numerics big-endian (the header stays v1-little-endian). v1 readers ignore the flag and run the package — trust enforcement is a reader capability, not a format barrier.

TABLE 3

offset	size	field
0	256	per-slot SHA-256, 8 × 32 B (digest of slot i at i×32; tail zeroed)
256	8	signer keyid (low 64 bits of OpenPGP fingerprint, BE)
264	sig_len	OpenPGP detached signature (binary packets)
264+sig_len	4	u32be sig_len (1..=65536)

Canonical signed bytes: `slot table ‖ ext blocks ‖ digests ‖ keyid ‖ header` — everything except the signature and its length field. The extension blocks (§5b) sit inside the signed region: the signature covers them.

© 2026 The Tebako Project

5. The installability flag (bit2 NO_INSTALL)

A run is a run — a plain execution never writes payload slices into the local store, with exactly one scoped exception: the lazy seed of spec 05 §4 (a self-contained package MAY seed the cache with the artifacts it carries — best-effort, never blocking the run). NO_INSTALL packages never seed. Installation is otherwise the EXPLICIT verb (`tebako install <path>`; the bootstrap's `--tebako-install` refuses or guides, spec 06 §4 exit 76). NO_INSTALL marks a publisher-frozen package: it RUNS standalone but every install attempt is refused with a named error. Absence means installable-on-request, including packages pressed before the verb existed — pass-through like every flag.

5a. The quiet-notices flag (bit3 QUIET_NOTICES; tebako#400, spec 23 §14)

The developer's declaration that the package's runs suppress the unsigned-legacy-trailer warning (spec 09 §v1-legacy) and the progress lines (spec 06 §5) — declared through any registry channel (CLI `--quiet-notices`, env TEBAKO_QUIET_NOTICES, compose key `quiet_notices:`), resolved once at press, baked here. Run-time reads only the bit: the package behaves identically on every machine. The bit rides inside the signed region, so it cannot be forged onto a signed package; TEBAKO_REQUIRE_SIGNED=1 is checked FIRST and outranks it, and the audit-journal entry is written regardless. Absence means the default notice behavior — pass-through like every flag.

6. Orthogonality note (normative)

format_id answers exactly one question: how to read the slot's bytes. Whether a slot is a runtime, and which payload carries the entrypoint, are roles — L2 semantics declared by manifests (spec 03), never encoded in the format axis. The v1 `format_id = 4 (TPKG_FORMAT_RUNTIME)` marks the fat runtime slot so v1 loaders skip handing it over as a mount; it is a role riding in the format field, kept only for backward compatibility. The two-slot era makes the wart obsolete in practice: a self-contained package carries its runtime as ordinary exe + env-image slots (spec 19 §6.1), and presses never emit format 4 anymore. The cleanup path: slot role moves to slot flags / package manifest, and format_id returns to pure image-format semantics. New semantics must never again be smuggled into the format axis.

© 2026 The Tebako Project

5b. Typed extension blocks (the L2 home; locked 2026-07-26)

Following OCI's model — the manifest is a separate blob BESIDE the layers, readable without pulling any image — the package-level (L2) manifest lives in the container but OUTSIDE every payload image. The v2 signing extension already proved the mechanism: blocks between the slot table and the trailer header are invisible to v1 readers (they only read the header at EOF and the slot table at its recorded offset; the bounds check never requires the table to abut the header).

Generalized layout:

FIGURE 2

```
[bootstrap][payload slots][slot table][ext blocks…][v2
signing ext?][header @EOF]

ext block: [u32be type][u32be length][payload bytes]
  type 1 = v2 signing extension (historical: immediately
before the
           header, self-delimiting from the tail via its sig_
len field)
  type 2 = package manifest (YAML — spec 03 §6; identity,
entrypoint/
           suite composition, package-level jail + env, per-
entry
           runtime refs, per-slot mount modes — exclusive/
union,
           spec 17 §1)
```

Rules: blocks walk forward from the end of the slot table (type+length self-delimit); the v2 signing ext, when present, is LAST before the header (unchanged bytes/position); unknown block types are skipped by v1-era-aware readers (forward-compat) but rejected by `validate` with a named error; a package manifest never duplicates payload manifests — it references slots, and payload manifests (spec 03) stay inside images (self-describing payloads). The 128-byte single `runtime_ref` remains for v1-era loaders; per-entry runtime references for suites and multi-runtime packages live in the type-2 manifest (no size cap).

6. Absent vs corrupt; validation

- Last-166-byte window without the `"TEBA"` prefix → `NoTrailer` (classic bundle; callers may fall back to offset auto-detection).

© 2026 The Tebako Project

- Prefix ok, magic bad → `Magic`. Magic ok, crc bad → `Crc`.
- Version ≠ 1 → `Version`. Slot count outside 1..=8 → `Slots`.
- Structural validation mirrors the reference C `tpkg_validate()`: `offset+size` non-overflowing, `format_id` ≤ 5 (spec 20 raised the bound from the C reference's ≤ 4 — the documented parity end: the C99 reader keeps rejecting format 5 with its named invalid error; ids ≥ 6 stay unallocated), `runtime_ref` and mount points NUL-terminated within their fixed fields; table fits entirely before the header.
- v2: flag/extension co-presence; zeroed digest tail; signature non-empty and ≤ 65536; signer keyid non-zero.

7. Errors (Rust `TpkgError` ↔ C `TPKG_ERR_*`, 1:1)

`NoTrailer`, `Magic`, `Crc`, `Io`, `Bounds`, `Slots`, `Invalid`, `Arg`, `Version` — `strerror` strings byte-identical with the C implementation.

© 2026 The Tebako Project